

My Project

Generated by Doxygen 1.13.2

Chapter 1

TOWER OF HANOI SOLVER

Minor project to solve the classic puzzle [Tower of Hanoi](#), based on 3 different wooden pegs where the main goal is to move a stack of n disks from an initial peg (peg A) to a target one (C), while using a spare peg (peg B). The rules to do so are the following:

1. Only one disk may be moved at a time.
2. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty peg.
3. No disk may be placed on top of a disk that is smaller than it.

The minimal number of moves required to solve a Tower of Hanoi puzzle is $2^n - 1$.

For example, for $n=3$:

```
Move number 0: A = {3, 2, 1}, B = {}, C = {}
Move number 1: A = {3, 2}, B = {}, C = {1}
Move number 2: A = {3}, B = {2}, C = {1}
Move number 3: A = {3}, B = {2, 1}, C = {}
Move number 4: A = {}, B = {2, 1}, C = {3}
Move number 5: A = {1}, B = {2}, C = {3}
Move number 6: A = {1}, B = {}, C = {3, 2}
Move number 7: A = {}, B = {}, C = {3, 2, 1}
```


Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[TowerHanoi](#) ??

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

TowerHanoi.hpp		??
--------------------------------	-------	----

Chapter 4

Class Documentation

4.1 TowerHanoi Class Reference

Public Member Functions

- **TowerHanoi** (int n)
- void **rec_solve** (int n, std::list< int > &origin, std::list< int > &spare, std::list< int > &target)
- void **printState** () const

Friends

- void **solve** ([TowerHanoi](#) &th)

The documentation for this class was generated from the following file:

- TowerHanoi.hpp

Chapter 5

File Documentation

5.1 TowerHanoi.hpp

```
00001 #ifndef TOWERHANOI_HPP
00002 #define TOWERHANOI_HPP
00003
00004 class TowerHanoi
00005 {
00006     private:
00007         int n_moves, n_disks;
00008         std::list<int> A, B, C; // Pegs are represented by lists
00009
00010     public:
00011
00012         // Constructor defined by the number of disks:
00013         TowerHanoi(int n) : n_moves(0), n_disks(n)
00014         {
00015             for (int i = 1; i <= n_disks; i++)
00016             {
00017                 A.push_front(i);
00018             }
00019             printState();
00020         }
00021
00022         // Solve the puzzle recursively
00023         void rec_solve(int n, std::list<int>& origin, std::list<int>& spare, std::list<int>& target)
00024         {
00025             if (n == 1)
00026             {
00027                 target.push_back(origin.back());
00028                 origin.pop_back();
00029                 n_moves++;
00030                 printState();
00031             }
00032             else
00033             {
00034                 rec_solve(n - 1, origin, target, spare);
00035                 target.push_back(origin.back());
00036                 origin.pop_back();
00037                 n_moves++;
00038                 printState();
00039                 rec_solve(n - 1, spare, origin, target);
00040             }
00041         }
00042
00043         // Print state after every move:
00044         void printState() const
00045         {
00046             std::cout << "Move number " << n_moves << ": ";
00047
00048             std::cout << "A = {";
00049             for (const auto& el : A)
00050             {
00051                 if (el == A.back())
00052                 {
00053                     std::cout << el;
00054                     break;
00055                 }
00056                 std::cout << el << ", ";
00057             }
00058         }
00059     };
00060 }
```

```
00058         std::cout << "}, ";
00059
00060         std::cout << "B = {";
00061         for (const auto& el : B)
00062         {
00063             if (el == B.back())
00064             {
00065                 std::cout << el;
00066                 break;
00067             }
00068             std::cout << el << ", ";
00069         }
00070         std::cout << "}, ";
00071
00072         std::cout << "C = {";
00073         for (const auto& el : C)
00074         {
00075             if (el == C.back())
00076             {
00077                 std::cout << el;
00078                 break;
00079             }
00080             std::cout << el << ", ";
00081         }
00082         std::cout << "}\n";
00083     }
00084
00085     // Solve friend function calling rec_solve:
00086     friend void solve(TowerHanoi& th);
00087 };
00088
00089 void solve(TowerHanoi& th)
00090 {
00091     th.rec_solve(th.n_disks, th.A, th.B, th.C);
00092 }
00093
00094 #endif
```